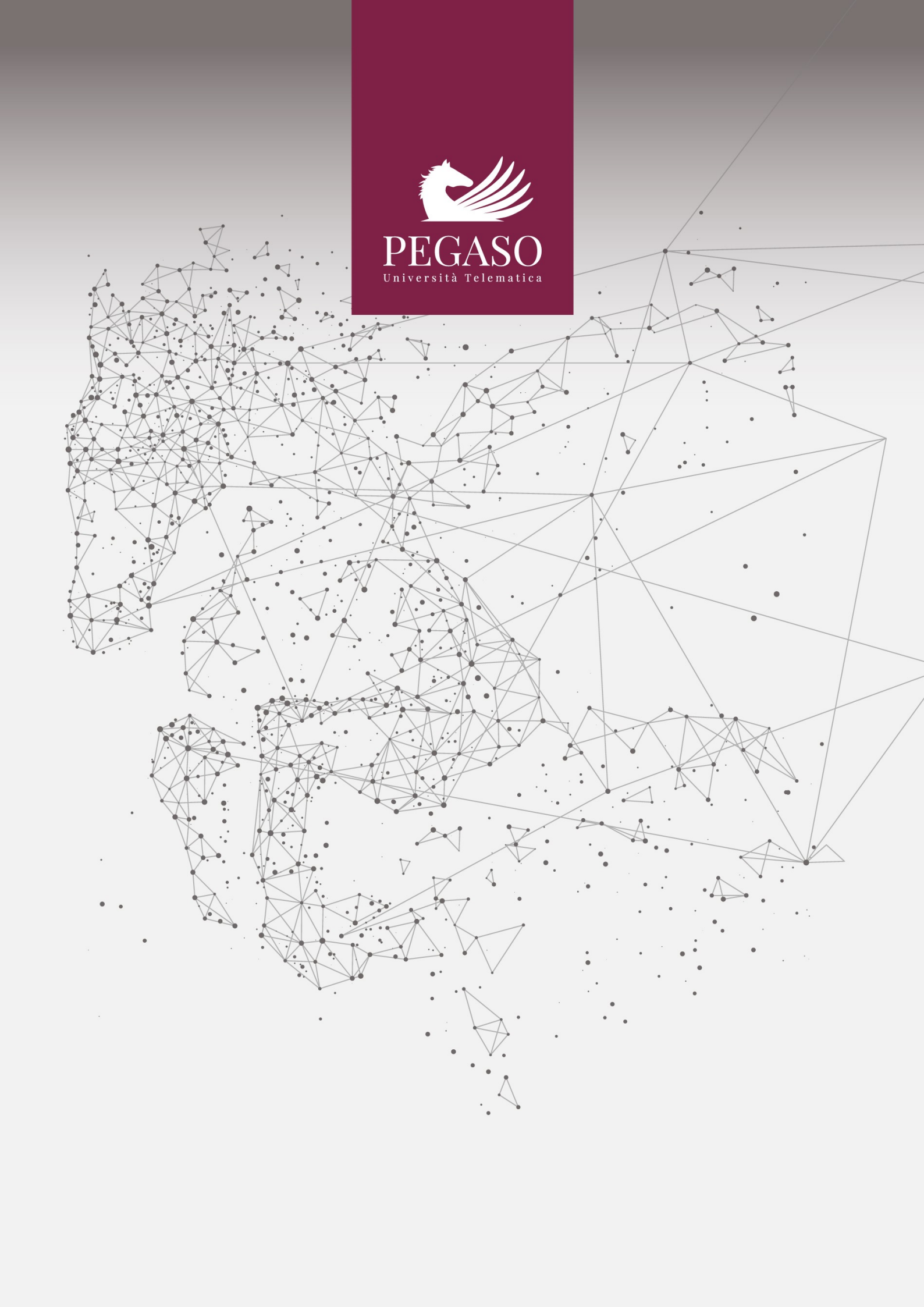

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. ARRAY IN C 3
2. DEFINIRE GLI ARRAY 6
3. ESEMPI DI ARRAY 7
4. INIZIALIZZAZIONE DI UN ARRAY IN UNA DEFINIZIONE CON UNA LISTA DI INIZIALIZZAZIONE 9
RIFERIMENTI BIBLIOGRAFICI 15

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Array in C

Gli array sono strutture di dati costituite da dati correlati e dello stesso tipo.

Più precisamente, un array è un gruppo di locazioni di memoria contigue che hanno tutte lo stesso tipo.

Per fare riferimento a una locazione o a un particolare elemento dell'array, specifichiamo:

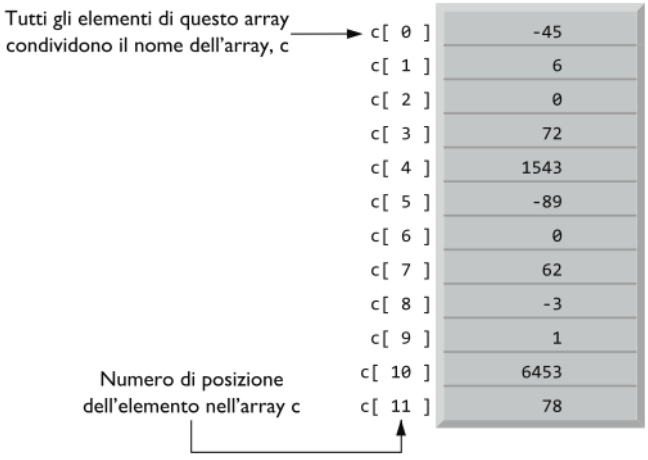
- il nome dell'array
- il numero di posizione dell'elemento nell'array.

La seguente figura mostra un array di interi chiamato c, contenente 12 elementi.

Si può fare riferimento a uno qualunque di questi elementi fornendo il nome dell'array seguito dal numero di posizione dell'elemento racchiuso tra parentesi quadre [].

Il primo elemento in ogni array è l'elemento di posizione zero (cioè quello con numero di posizione 0).

Il nome di un array, come altri identificatori, può contenere soltanto lettere, cifre e sottolineature e non può iniziare con una cifra.



Il numero di posizione tra le parentesi quadre è chiamato indice dell'elemento.

Un indice deve essere un intero o un'espressione intera.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ad esempio, l'istruzione

```
c[2] = 1000;
```

assegna 1000 all'elemento dell'array `c[2]`.

In modo simile, se `a = 5 e b = 6`, l'istruzione

```
c[a + b] += 2;
```

addiziona 2 all'elemento dell'array `c[11]`.

Un nome di array con indice è un *l-value* (locator value) che può essere usato sul lato sinistro di un'assegnazione.

Analizziamo più da vicino l'array `c`.

Il nome dell'array è `c`.

I suoi 12 elementi sono chiamati `c[0], c[1], c[2], ..., c[10] e c[11].`

Il valore memorizzato in `c[0]` è -45, il valore di `c[1]` è 6, di `c[2]` è 0, di `c[7]` è 62 e di `c[11]` è 78.

Per stampare la somma dei valori contenuti nei primi tre elementi dell'array c scriviamo

```
printf("%d", c[0] + c[1] + c[2]);
```

Per dividere il valore dell'elemento 6 dell'array `c` per 2 e assegnare il risultato alla variabile x scriviamo

```
x = c[6] / 2;
```

Le parentesi usate per racchiudere l'indice di un array sono in realtà considerate in C un operatore.

Esse hanno lo stesso livello di precedenza dell'operatore di chiamata di funzione (cioè le parentesi poste dopo il nome di una funzione nella chiamata di quella funzione).

La figura seguente mostra la precedenza e l'associatività degli operatori introdotti fino a questo punto nel testo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Operatori	Associatività	Tipo
`[]` `()` `++` (*postfisso*) `--` (*postfisso*)	da sinistra a destra	precedenza più alta
`+` `-` `!` `++` (*prefisso*) `--` (*prefisso*) (*tipo*)	da destra a sinistra	unario
`*` `/` `%`	da sinistra a destra	moltiplicativo
`+` `-`	da sinistra a destra	additivo
`<` `<=` `>` `>=`	da sinistra a destra	relazionale
`==` `!=`	da sinistra a destra	di uguaglianza
`&&`	da sinistra a destra	AND logico
`||`	da sinistra a destra	OR logico
`?:`	da destra a sinistra	condizionale
`=` `+=` `-=` `*=` `/=` `%=`	da destra a sinistra	di assegnazione
`,`	da sinistra a destra	virgola

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Definire gli array

Gli array occupano spazio in memoria.

Dovete specificare il tipo di ogni elemento e il numero di elementi che ogni array richiede, in modo che il computer possa riservare la giusta quantità di memoria.

La definizione seguente riserva 12 elementi per l'array di interi c, che ha indici nell'intervallo da 0 a 11.

```
int c[12];
```

La definizione

```
int b[100], x[27];
```

riserva 100 elementi per l'array intero b e 27 elementi per l'array intero `x`.

Questi array hanno indici, rispettivamente, negli intervalli da 0 a 99 e da 0 a 26.

Sebbene sia possibile definire più array in una volta, è preferibile definirne solo uno per riga in modo da poter aggiungere un commento che spieghi lo scopo di ciascuno di essi.

Gli array possono contenere altri tipi di dati.

Ad esempio, un array di tipo char può memorizzare una stringa di caratteri.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Esempi di array

Definizione di un array e uso di un ciclo per impostare i valori degli elementi di un array

Come le altre variabili, gli elementi di un array non inizializzati contengono valori spazzatura.

Il seguente programma usa l'istruzione for per impostare gli elementi di un array n intero di 5 elementi con valori uguali a zero (righe 11–13) e per stampare l'array in formato tabellare (righe 18–20).

La prima istruzione `printf` (riga 15) stampa le intestazioni per le due colonne stampate nella successiva istruzione `for`.

```
1  // Programma in C per
2  // inizializzare gli elementi di un array a zero.
3  #include <stdio.h>
4
5  // la funzione main inizia l'esecuzione del programma
6  int main(void)
7  {
8     int n[5]; // n e' un array di 5 interi
9
10    // imposta gli elementi dell'array n a 0
11    for (size_t i = 0; i < 5; ++i) {
12       n[i] = 0; //imposta 0 l'elemento alla locazione i
13    }
14
15    printf("%s%13s\n", "Element", "Value");
16
17    // invia in uscita i contenuti dell'array n
18    for (size_t i = 0; i < 5; ++i) {
19       printf("%7u%13d\n", i, n[i]);
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
20      }
21  }
```

Element	Value
0	0
1	0
2	0
3	0
4	0

Notate che la variabile di controllo contatore i è dichiarata di tipo `size_t` in ogni istruzione `for` (righe 11 e 18), che secondo il C standard rappresenta un tipo intero senza segno[1] .

Questo tipo è raccomandato per qualsiasi variabile che rappresenta la dimensione o gli indici di un array.

Il tipo è definito nell'intestazione `<stddef.h>`, che spesso è inclusa da altre intestazioni (come `<stdio. h>`) .

Nota: se provando a compilare il programma precedente ricevete la segnalazione di errori, includete semplicemente `<stddef.h>` nel vostro programma.

[1] Su alcuni compilatori, `size_t` rappresenta `unsigned int` e su altri rappresenta `unsigned long`. I compilatori che utilizzano `unsigned long` solitamente generano un avvertimento sulla riga 19 del programma, perché `%u` è per la stampa di `unsigned int`, non di `unsigned long`. Per eliminare questo avvertimento, sostituite la specificazione di formato `%u` con `%lu`.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Inizializzazione di un array in una definizione con una lista di inizializzazione

Gli elementi di un array possono essere inizializzati anche quando l'array viene definito, facendo seguire la definizione dal segno di uguale e da parentesi graffe, { }, contenenti una lista di inizializzatori dell'array separati da virgole.

Il seguente programma inizializza un array intero con cinque valori (riga 9) e stampa l'array in formato tabellare.

```
01 // Programma in C per
02 // Inizializzare un array con lista di inizializzazione
03 #include <stdio.h>
04
05 // la funzione main inizia l'esecuzione del programma
06 int main(void)
07 {
08    // usa una lista per inizializzare l'array n
09    int n[5] = {32, 27, 64, 18, 95};
10
11    printf("%s%13s\n", "Element", "Value");
12
13    // invia in uscita i contenuti dell'array
14    for (size_t i = 0; i < 5; ++i) {
15       printf("%7u%13d\n", i, n[i]);
16    }
17 }
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Element	Value
0	32
1	27
2	64
3	18
4	95

Se vi sono meno inizializzatori rispetto agli elementi dell'array, i restanti elementi sono inizializzati a zero.

Ad esempio, gli elementi dell'array n avrebbero potuto essere inizializzati a zero come segue:

```
int n[10] = {0}; // inizializza l'intero array a zero
```

Questa definizione inizia esplicitamente il primo elemento a zero e inizializza anche i restanti nove elementi a zero perché vi sono meno inizializzatori di quanti sono gli elementi nell'array.

Gli array non sono automaticamente inizializzati a zero.

Dovete almeno inizializzare il primo elemento a zero perché i restanti elementi siano automaticamente azzerati.

Gli elementi dell'array sono inizializzati prima dell'avvio del programma per gli array static e al momento dell'esecuzione per gli array automatici.

☹ *Errore comune di programmazione*

Dimenticare di inizializzare gli elementi di un array.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

☹ *Errore comune di programmazione*

Fornire una lista con più inizializzatori per un array di quanti siano gli elementi dell'array è un errore di sintassi. Ad esempio, int n[3] = {32, 27, 64, 18}; è un errore di sintassi, poiché ci sono quattro inizializzatori ma solo tre elementi dell'array.

Se la dimensione dell'array è omessa in una definizione con una lista di inizializzatori, il numero di elementi dell'array sarà il numero di elementi nella lista.

Ad esempio,

```
int n[] = {1, 2, 3, 4, 5};
```

crea un array di cinque elementi inizializzati con i valori indicati.

Specificare la dimensione di un array con una costante simbolica e inizializzare gli elementi dell'array con espressioni da calcolare

Il seguente programma inizializza gli elementi di un array `s` di 5 elementi con i valori 2, 4, 6, 8, 10 e stampa l'array in formato tabellare.

I valori sono generati moltiplicando il contatore del ciclo per 2 e aggiungendo 2.

```
1  // Programma in C che inizializza gli elementi
2  // dell'array s con gli interi pari da 2 a 10.
3  #include <stdio.h>
4  #define SIZE 5 // dimensione dell'array
5
6  // la funzione main inizia l'esecuzione del programma
7  int main(void)
8  {
9  // la costante SIZE specifica la dimensione dell'array
10    int s[SIZE]; // l'array ha un numero SIZE di elementi
11
12    for (size_t j = 0; j < SIZE; ++j) {
13       s[j] = 2 + 2 * j;
14    }
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
15
16      printf("%s%13s\n", "Element", "Value");
17
18      // invia in uscita i contenuti dell'array s
19      for (size_t j = 0; j < SIZE; ++j) {
20         printf("%7u%13d\n", j, s[j]);
21      }
22   }
```

Element	Value
0	2
1	4
2	6
3	8
4	10

In questo programma viene introdotta la **direttiva per il preprocessore** `#define`.

La riga 4:

```
#define SIZE 5
```

definisce una costante simbolica `SIZE` il cui valore è 5.

Una **costante simbolica** è un identificatore che è sostituito dal preprocessore del C con un testo di sostituzione prima che il programma sia compilato.

Quando il programma è preprocessato, la costante simbolica `SIZE` viene sostituita ogni volta che ricorre con il testo di sostituzione 5.

L'uso di costanti simboliche per specificare le dimensioni di un array rende i programmi più modificabili.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nel programma potremmo far riempire un array di 1000 elementi con il primo ciclo `for` (riga 12) cambiando semplicemente il valore di `SIZE` nella direttiva `#define` da 5 a 1000.

Se non venisse usata la costante simbolica `SIZE`, dovremmo modificare il programma nelle righe 10, 12 e 19.

Quando i programmi diventano grandi, questa tecnica diventa molto utile per scrivere programmi chiari e di facile lettura e manutenzione.

Una costante simbolica (come `SIZE`) è più facile da comprendere del valore numerico 5, che potrebbe avere diversi significati all'interno del codice.

☺ *Errore comune di programmazione*

Terminare una direttiva per il preprocessore `#define` o `#include` con un punto e virgola.

Ricordate che le direttive per il preprocessore non sono istruzioni del C.

Se la direttiva per il preprocessore `#define` nella riga 4 viene terminata con un punto e virgola, il preprocessore sostituisce con il testo "5;" la costante simbolica `SIZE` tutte le volte che ricorre nel programma.

Questo può portare a errori di sintassi in fase di compilazione o a errori logici in fase

di esecuzione.

Ricordate che il preprocessore non è il compilatore del C.

☺ *Osservazione di ingegneria del software*

Definire la dimensione degli array con costanti simboliche rende i programmi più modificabili.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

☹ *Errore comune di programmazione*

Assegnare un valore a una costante simbolica in un'istruzione eseguibile è un errore di sintassi.

Il compilatore non riserva spazio per le costanti simboliche come fa per le variabili che memorizzano valori in fase di esecuzione.

☺ *Buona pratica di programmazione*

Usate solo lettere maiuscole per i nomi di costanti simboliche. Questo evidenzia tali costanti in un programma e vi ricorda che le costanti simboliche non sono variabili.

☺ *Buona pratica di programmazione*

Nei nomi di costanti simboliche formati da più parole, separate le parole con sottolineature per favorirne la leggibilità.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Riferimenti bibliografici

- Paul Deitel, Harvey Deitel, "Il linguaggio C – Fondamenti e tecniche di programmazione", Libro edito da Pearson Italia. Include anche utili esercizi di autovalutazione.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).